

Wikipedia Android App 软件测试大作业 报告

课程：移动应用测试

指导教师：胡燕

组号：1

完成日期：2026-06-17

Github 仓库地址：<https://github.com/mvp2468/-/.git>

表 1 小组成员及任务分工表

小组成员	学号	班级	任务分工
彭博	20232241037	软件 2305	集成测试、报告
赵浩然	20232241289	软件 2306	单元测试、报告
田红森	20231011028	软件 2306	报告、PPT 制作、课堂汇报
吴予哲	20232241484	软件 2309	系统测试

大连理工大学

Dalian University of Technology

摘要

本报告以开源 Android 项目 `apps-android-wikipedia-main` 维基百科客户端为被测对象，遵循软件测试分层思想，完整开展单元测试、集成测试、系统测试全流程实践。采用 JUnit+Robolectric 完成底层单元自动化测试，Espresso+Compose Testing 实现真机集成 UI 测试，结合人工场景法完成系统黑盒功能与性能测试。项目共执行 1326 条单元用例、120 条集成自动化用例、22 条系统业务场景，全部用例执行通过；单元与集成测试合并综合行覆盖率 44.0%，分支覆盖率 23.5%。报告分析了三层测试的实施过程、执行数据、现存缺陷，针对项目架构可测性差、覆盖率不均衡、内存线程性能隐患给出短期、中期、长期分层优化方案，完整验证分层测试在移动 App 质量保障中的作用。

一、概述

1.1 项目简介

本次被测项目为开源 Android 客户端 `apps-android-wikipedia-main`（维基百科 App），采用传统 XML 视图+Jetpack Compose 混合开发，整体分层架构分为网络数据层、本地 Room 持久层、ViewModel 业务层、UI 视图层、通用工具层。业务覆盖多语言词条搜索、文章图文浏览、Explore 信息流、阅读/监视收藏列表、词条分类与用户讨论、Wiktionary 词典弹窗、软件设置、词条在线编辑等完整功能。

1.2 测试分层方案

按照软件测试标准分层体系，分为**单元测试**、**集成测试**、**系统测试**三大部分开展全流程验证，三层测试各司其职形成质量闭环：

1. 单元测试：聚焦底层独立代码单元（工具类、数据模型、数据库转换器），本地运行无需设备，验证纯逻辑正确性；
2. 集成测试：基于真机执行 Instrumented 自动化 GUI 测试，验证 Activity、Fragment、数据库、网络组件之间跨模块交互逻辑；
3. 系统测试：黑盒人工全场景测试，将应用作为完整产品，模拟普通用户完整业务操作链路，同时补充 CPU、内存非功能性能测试。

1.3 测试工具与环境

1. 硬件设备：PHV110 真机，Android 14 系统，USB 调试模式；
2. 单元测试技术栈：JUnit4、Robolectric、Mockito、JaCoCo 覆盖率统计工具；
3. 集成测试技术栈：Espresso、Compose Testing、UiAutomator；
4. 系统测试辅助工具：Appium（控件定位）、Android Studio Profiler（性能监

控)；

5. 项目构建工具：Gradle，提供批量执行测试、生成 HTML 覆盖率报表命令。

1.4 整体测试概况

项目共计 150 个单元测试文件、1326 条单元测试用例；43 个集成测试文件、约 120 条 UI 自动化用例；设计 22 条人工系统功能测试场景。三层所有测试用例全部执行通过，无失败阻塞缺陷。单元测试与集成测试覆盖率合并统计，综合行覆盖率 44.0%，分支覆盖率 23.5%。

二、单元测试

2.1 测试目标与覆盖范围

单元测试针对程序**最小可独立运行代码单元**，通过 Mock 手段隔离网络、数据库、Android 运行时等外部依赖，仅校验内部纯业务计算逻辑，不涉及页面交互、组件渲染。

本次单元测试覆盖模块：

1. dataclient 下全部网络请求实体、API 响应数据模型；
2. Room 数据库 TypeConverter 类型转换器；
3. util 通用工具类（日期处理、字符串解析、图片地址转换、网络状态判断）；
4. OkHttp 网络拦截器内部缓存逻辑、CSRF 令牌工具、埋点统计分析类。

2.2 测试设计思路

1. 采用 Robolectric 轻量化模拟 Android 系统环境，无需连接真机，本地快速执行；
2. 使用 Mockito 模拟数据库实例、网络请求返回数据，消除外部依赖干扰；
3. 用例设计覆盖正常流程、空参数、异常 JSON、边界数值、异常分支等多场景；
4. 执行速度快，适合开发迭代过程中持续回归底层基础逻辑。

2.3 单元测试执行数据

表 2 单元测试核心执行指标统计表

指标	数值
单元测试文件总数	150 个
单元测试方法总数	1326 个
测试通过率	100%（1326/1326）
整套单元测试执行耗时	2 分 42 秒
仅单元测试行覆盖率	8.9%
仅单元测试分支覆盖率	3.8%

2.4 典型单元测试用例类型

- 1. 数据模型序列化测试：校验 JSON 正反解析逻辑、空字段、异常数据容错能力；
- 2. 时间统计计算测试：阅读量统计、增长任务不同日期区间计算结果校验；
- 3. 数据库类型转换测试：LocalDateTime、通知类型、命名空间双向序列化转换；
- 4. 网络缓存策略测试：区分在线、离线、缓存过期三种场景下缓存判断逻辑。

2.5 单元测试优缺点与局限

- 1. 优势：执行速度快、缺陷定位精准、可集成 CI 频繁回归、不依赖硬件设备；
- 2. 局限：仅能覆盖无界面底层逻辑，无法覆盖 Activity、Compose 页面、页面跳转、数据库真实读写逻辑；UI 层、跨模块交互代码完全无法覆盖，单独运行覆盖率极低，必须搭配集成测试补充覆盖。

2.6 高覆盖单元模块说明

dataclient.watch、dataclient.growthtasks、dataclient.restbase、staticdata 四类数据层包单元覆盖率均超过 85%，所有数据实体模型均完成完整单元校验，底层数据解析、计算逻辑稳定性高。

三、集成测试

3.1 测试目标与覆盖范围

集成测试为本项目 Instrumented GUI 自动化测试，必须在真机/模拟器环境运行，

核心验证多个模块、组件之间协同交互逻辑，弥补单元测试无法覆盖页面、视图、数据库联动交互的短板。

重点验证场景：Activity 与 Fragment 页面联动、Compose 组件与 ViewModel 数据渲染、页面与 Room 数据库真实读写、网络请求加载后页面渲染、底部标签切换、弹窗组件展示逻辑。

3.2 测试设计思路

- 1. 使用 Espresso 操作传统 XML 布局控件，Compose Testing 专门测试 Jetpack Compose 弹窗与页面；
- 2. 自动化脚本模拟用户点击、滑动、切换标签、等待接口加载等真实操作；
- 3. 本次作业新增 5 套专项集成测试类，补齐讨论模块、监视列表、分类页面、词典弹窗、数据库交互模块的测试空白；
- 4. 统一执行集成测试套件 IntegrationTestSuite，批量自动化校验页面交互流程。

3.3 集成测试执行数据

- 1. Instrumented 集成测试文件：43 个，分布于文章、分类、讨论、监视列表、设置、信息流等目录；
- 2. UI 自动化测试方法总量：约 120 个；
- 3. 标准集成测试套件：11 个测试用例全部通过。

表 3 新增集成测试类覆盖明细表

测试类	测试方法	覆盖场景	覆盖率增量
AppDatabaseTests	4	数据库增删查与页面联动交互	1.80%
CategoryActivityTest	2	分类页面加载、子分类标签切换	5.80%
TalkTopicsActivityTest (新增)	3	讨论话题列表加载、点击跳转详情页	5.43%
WatchlistActivityTest (新增)	2	监视列表页面启动、空状态页面渲染	2.65%
WiktionaryDialogComposeTest	4	词典弹窗加载、错误、成功、多释义渲染	1.62%

3.4 单元+集成合并覆盖率结果

表 4 单元与集成测试合并覆盖率对比表

覆盖维度	行覆盖率	分支覆盖率
仅本地单元测试	8.9%	3.8%
仅集成 UI 自动化测试	34.2%	17.5%
单元+集成基础合并覆盖率	34.5%	17.0%
新增 5 套集成测试增量覆盖	+9.5%	+6.5%
项目最终综合全量覆盖率	44.0%	23.5%

3.5 集成测试关键发现

- 1. 大量 UI 视图相关包单元测试覆盖率为 0，经过集成测试补充后覆盖率提升 70%~94%，包含信息流、底部导航栏、离线数据库、图片加载服务等模块；
- 2. 自动化测试现存执行阻碍：新版本临时账户功能阻断 Diff 差异页面原有导航流程、文章编辑详情页面 API 加载超时、开发环境偶现测试驱动中断；
- 3. 项目共有 42 个业务包覆盖率为 0，合计 6740 行代码，占总代码 10.8%；其中 categories、categories.db、wiktionary 已新增集成测试，重新生成覆盖率报表后可脱离零覆盖清单。

3.6 集成测试价值

集成测试是提升项目整体代码覆盖率的核心手段，解决单元测试无法覆盖视图层、页面交互的痛点；可提前发现页面、数据库、网络多模块联动产生的兼容性缺陷，保障各业务模块组装后能够正常协同工作。

四、系统测试

4.1 测试目标与覆盖范围

系统测试属于黑盒全流程测试，不关注程序内部代码实现，将 App 作为完整独立产品，模拟普通用户真实操作行为，验证完整业务流程、页面展示、边界场景可用性。

覆盖全部核心业务模块：词条搜索、文章浏览、Explore 信息流、底部导航、收藏阅读列表、端到端完整用户操作旅程。

4.2 测试设计思路

采用场景法设计功能用例，借助 Appium 识别页面控件，人工在 Android14 真机逐条执行验证；每条用例规范包含操作步骤、预期结果、实际运行结果、测试结论四项内容。

4.3 系统测试用例统计

本次共设计 5 大业务模块，合计 22 条独立测试场景，**全部执行通过，通过率 100%**：

1. 搜索功能（4 条）：英文词条搜索、中文词条搜索、无结果词条搜索、输入联想推荐校验；
2. 文章浏览（5 条）：打开词条、页面上下滚动、目录跳转、收藏保存文章、返回首页；
3. Explore 信息流（4 条）：页面加载、下拉刷新、精选文章卡片跳转、上滑加载更多卡片；
4. 底部导航栏（5 条）：五大标签切换、更多菜单展开、设置页面入口、来回切换页面、收藏列表展示；
5. 完整用户旅程（4 条端到端场景）：搜索阅读流程、收藏查看流程、设置页面多级跳转、信息流阅读流程。

4.4 系统测试功能核心结论

1. 全业务流程无崩溃、闪退、页面跳转异常等阻断性缺陷；
2. 边界场景适配完善：无搜索结果、空收藏列表、长图文滚动均无卡顿；
3. 多语言输入搜索适配正常，页面来回切换不会丢失搜索记录、阅读进度；
4. 当前版本无严重功能缺陷，可满足普通用户基础使用需求。

4.5 配套系统级性能测试

在功能系统测试基础上，开展 20 分钟前台稳态性能监控，属于系统非功能测试：

1. CPU 指标：长期稳定运行仅占用 2%CPU，功耗控制优秀；但活跃线程总数 128 条，存在大量闲置后台线程，存在资源浪费；
2. 内存指标：应用总占用 361.5MB，设备分配上限 384MB，剩余安全余量 22.5MB，未检测到内存泄漏；但 Native、图形缓存、本地 Sqlite 缓存占用过高，高频浏览图文词条存在 OOM 崩溃隐患；
3. 堆快照分析：0 处 Activity/Fragment 生命周期泄漏，页面销毁逻辑规范；缺少缓存自动清理机制，缓存会持续累积占用内存。

五、综合总结

5.1 三层测试成果汇总

1. 单元测试：完成 1326 条底层逻辑用例，全覆盖数据模型与工具类，保障底层计算、序列化、类型转换逻辑稳定，是迭代回归的基础保障；缺点是仅能覆盖少量底层代码，无法验证页面交互；

2. 集成测试：通过 43 套 UI 自动化用例补齐页面、组件、数据库交互逻辑，直接将项目综合覆盖率提升至 44%，是本项目质量保障核心环节；同时识别出页面交互、自动化脚本执行存在的兼容性问题；

3. 系统测试：22 条人工业务场景全量通过，从真实用户视角验证 App 整体可用性；同步完成 CPU、内存系统级性能检测，定位线程冗余、缓存堆积等非功能隐患。

三层测试形成完整质量闭环：单元测试兜底底层逻辑、集成测试保障多模块交互、系统测试验证完整产品功能与性能，三者缺一不可。

5.2 项目现存主要问题

1. 架构可测性缺陷：全局静态单例泛滥、Hilt 依赖注入落地不全，大量代码无法隔离依赖，大幅提升单元测试编写成本；

2. 代码覆盖不均衡：42 个业务包完全零覆盖，大量 Compose 界面缺少专项集成测试；

3. 性能隐患：闲置后台线程过多、图片与本地缓存无容量上限、无自动过期清理机制；

4. 自动化测试稳定性不足：部分页面受临时账户、网络状态影响，脚本执行容易失败。

5.3 分层优化改进方案

1. 短期低成本优化（1-2 周）：补充事件数据类 JSON 单元测试、完善现有集成测试异常分支，快速提升模块覆盖率；

2. 中期重构优化（3-4 周）：拆分网络拦截器职责、工具类改造为构造注入、为零覆盖模块新增集成自动化测试；

3. 长期架构优化（版本迭代落地）：全项目统一 Hilt 依赖注入、统一全局协程线程调度、规范缓存容量与清理策略、过滤 Room/Hilt 自动生成代码的覆盖率统计。

5.4 课程学习总结

通过完整落地单元、集成、系统三层测试，清晰区分三层测试的定位与适用场景：单元测试面向最小代码单元、集成测试面向模块间交互、系统测试面向完整产品

业务。实践证明，仅依靠单元测试无法保障移动 App 整体质量，必须搭配集成自动化测试补齐视图层代码覆盖，再通过系统测试模拟真实用户体验，才能全方位把控软件功能、稳定性、性能整体质量。

附录

附录 A 测试执行 Gradle 核心命令

```
# 执行全量本地单元测试
./gradlew testDevDebugUnitTest
# 生成单元测试覆盖率报告
./gradlew jacocoTestReport
# 运行真机集成 UI 自动化测试
./gradlew connectedDevDebugAndroidTest
# 生成单元+集成合并覆盖率报告
./gradlew jacocoCombinedReport
```